



Baocao - 3123123

Lập trình game (Học viện Công nghệ Bưu chính Viễn thông)



messages.pdf_cover_qr_code_label

TRƯỜNG ĐẠI HỌC KIẾN TRÚC HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN

*** * ***



TIỂU LUẬN MÔN:
LẬP TRÌNH GAME

Đề tài : Thiết kế game Dnd 2D Unity

Giảng viên hướng dẫn: ThS. Nguyễn Mạnh Hùng

Lớp: 21CĐP2

Nhóm 10 :

1. Nguyễn Tài Thu - 2155020092 (Nhóm trưởng)
2. Mai Ngọc Quyến - 2155020080
3. Trần Việt Hoàng - 2155020034
4. Phạm Đức Trung - 2155020104
5. Phạm Thế Anh - 2155020008

Hà Nội, 6/2024

Mục lục

Lời mở đầu.....	3
Chương 1: Tổng quan.....	4
I. Lý do lựa chọn đề tài:.....	4
II. Tình hình thực tế và yêu cầu:.....	4
III. Mục tiêu nghiên cứu:.....	4
a. Mục tiêu:.....	4
b. Nhiệm vụ nghiên cứu:.....	5
c. Đối tượng và phạm vi nghiên cứu:.....	5
d. Phương pháp nghiên cứu:.....	5
Chương 2: Cơ sở lý thuyết.....	6
I. Tổng quan về Unity Engine:.....	6
1.1. Unity là gì:.....	6
1.2. Quá trình phát triển:.....	7
1.3. Một số thống kê về Unity:.....	7
1.4. Ưu điểm của Unity:.....	7
II. Các thành phần trong Unity:.....	8
2.1. Assets:.....	8
2.2. Scenes:.....	9
2.3. Game Objects:.....	9
2.4. Components:.....	9
2.5. Scripts:.....	10
2.6. Prefabs:.....	11
2.7. Material và Shader:.....	11
2.8. UnityAPI:.....	11
2.9. Không gian 3D và vector trong không gian:.....	12
III. Sơ lược về giao diện Unity:.....	13
3.1. Giao diện:.....	13
3.2. Cửa sổ Scene và Hierarchy:.....	13
3.3. Cửa sổ Inspector:.....	14

3.4. Cửa sổ Project:.....	15
3.5. Cửa sổ Game:.....	16
IV. Tìm hiểu về công cụ hỗ trợ :.....	16
Chương 3: Mô tả thiết kế game.....	20
I. Thiết kế:.....	20
1. Cốt truyện:.....	20
1.2. Game Setting:.....	20
II. Cơ chế và Tiến trình chơi:.....	26
2.1. Tiến trình trò chơi:.....	26
2.2. Cơ chế chơi:.....	27
Chương 4: Xây dựng Game và Demo.....	29
I. Xử lí di chuyển, va chạm, quái, kết thúc màn chơi:.....	29
II. Demo:.....	34
2.1. Gamplay:.....	34
2.2 Demo trò chơi:.....	34
III. Các chức năng cơ bản:.....	38
Chương 5: Kết luận và định hướng phát triển.....	40
I. Kết quả đạt được:.....	40
II Kết luận:.....	40
III. Định hướng phát triển:.....	40
Bảng phân chia công việc.....	41

Lời mở đầu

Trong thời đại công nghệ số hiện nay, ngành công nghiệp trò chơi điện tử đã và đang phát triển mạnh mẽ, trở thành một phần không thể thiếu trong cuộc sống giải trí của con người. Các trò chơi không chỉ đơn thuần mang tính giải trí mà còn là cầu nối giúp con người thể hiện sự sáng tạo, tư duy logic và khả năng làm việc nhóm.

Trò chơi nhập vai (RPG - Role-Playing Game) là một thể loại game phổ biến và được yêu thích bởi khả năng đưa người chơi vào những câu chuyện hấp dẫn, thế giới giả tưởng phong phú và cho phép họ trở thành những nhân vật mà họ yêu thích. Để hiện thực hóa một trò chơi RPG, việc lựa chọn nền tảng và công cụ phát triển phù hợp là điều vô cùng quan trọng. Unity, một công cụ phát triển game mạnh mẽ và phổ biến, đã được nhóm em lựa chọn cho đề án này.

Đề án phát triển trò chơi RPG 2D bằng Unity nhằm mục đích tạo ra một trò chơi nhập vai đơn giản nhưng đầy đủ các yếu tố cơ bản của thể loại này, từ việc xây dựng cốt truyện, thiết kế nhân vật, lập trình cơ chế chiến đấu, cho đến việc tạo dựng môi trường và các nhiệm vụ trong game. Qua đó, đề án cũng là cơ hội để nhóm áp dụng những kiến thức đã học, trau dồi kỹ năng lập trình, tư duy logic và khả năng làm việc nhóm.

Nhóm em hy vọng rằng, thông qua đề án này, không chỉ nâng cao được kỹ năng và kiến thức chuyên môn mà còn đóng góp một phần nhỏ vào kho tài liệu tham khảo cho những ai có cùng đam mê và hướng phát triển trong lĩnh vực lập trình game. Nhóm em xin chân thành cảm ơn sự hướng dẫn và hỗ trợ từ các giảng viên, bạn bè và gia đình trong suốt quá trình thực hiện đề án này.

Do kiến thức còn hạn chế nên bài nghiên cứu không tránh khỏi những thiếu sót, rất mong nhận được ý kiến đóng góp của thầy cô và các bạn để bài của nhóm chúng em được hoàn thiện hơn.

Chúng em xin chân thành cảm ơn!

Chương 1: Tổng quan

I. Lý do lựa chọn đề tài:

Thị trường game tại Việt Nam đang phát triển và có tiềm năng lớn. Trong thời đại công nghệ thông tin như hiện nay, sản phẩm công nghệ ngày càng chịu sự đánh giá khắt khe hơn từ phía những người dùng, đặc biệt là về sản phẩm Game được nhận rất nhiều sự đánh giá từ phía các Game thủ, hay chỉ là những người chơi bình thường. Ngành công nghiệp Game hiện nay có thể nói là bùng nổ, với tốc độ phát triển đến chóng mặt, rất nhiều những Game hay và hấp dẫn đã được ra đời trong thời gian qua. Vì thế nhóm em chọn đề tài này để xây dựng game đưa đến cho người dùng trải nghiệm và tìm hiểu kỹ hơn về xây dựng game.

II. Tình hình thực tế và yêu cầu:

Tình hình thực tế:

Ngành công nghiệp game đang trải qua sự phát triển mạnh mẽ, đặc biệt là trong lĩnh vực game di động và PC/console. Unity đã trở thành một trong những nền tảng phát triển game phổ biến nhất nhờ vào tính linh hoạt và khả năng hỗ trợ đa nền tảng.

Unity có một cộng đồng lớn và sôi động, cung cấp nhiều tài liệu học tập, hướng dẫn và hỗ trợ từ cộng đồng để giúp các nhà phát triển vượt qua các thử thách trong quá trình phát triển game.

Yêu cầu:

- **Xây dựng game hoàn chỉnh:** Hoàn thành một trò chơi RPG 2D với các yếu tố cơ bản như nhân vật, bản đồ, hệ thống điểm số, và các màn chơi.
- **Hiệu ứng và âm thanh:** Tạo ra các hiệu ứng hình ảnh và âm thanh phong phú để làm tăng tính hấp dẫn của trò chơi.
- **Tìm hiểu và áp dụng C# và Unity:** Nắm vững ngôn ngữ lập trình C# và áp dụng các kiến thức này vào việc phát triển game trên Unity.
- **Thử nghiệm và điều chỉnh:** Thử nghiệm và điều chỉnh game để đảm bảo tính hoàn thiện và trải nghiệm người chơi tốt nhất có thể.

III. Mục tiêu nghiên cứu:

a. Mục tiêu:

- Xây dựng được game hoàn chỉnh với các yếu tố cơ bản của một trò chơi RPG 2D như hệ thống nhân vật, bản đồ, hệ thống điểm số, và các màn chơi khác nhau.
- Tạo ra các hiệu ứng hình ảnh và âm thanh phong phú để làm tăng tính thú vị và trải nghiệm người chơi.
- Tìm hiểu về ngôn ngữ lập trình C#, đối tượng hỗ trợ C# và Unity.

b. Nhiệm vụ nghiên cứu:

- Tìm hiểu và áp dụng các kỹ thuật xây dựng kịch bản game trên nền tảng Unity, từ việc thiết kế các nhân vật cho đến cách xử lý va chạm và tạo hình ảnh động (animation).

c. Đối tượng và phạm vi nghiên cứu:

Đối tượng nghiên cứu:

- Lập trình game trên Unity.

Nền tảng unity engine:

- Cách thiết kế nhân vật, tạo bản đồ, khung cảnh, character animator, xử lý va chạm, health, player và enemy, điểm số, màn chơi, tạo AI cho enemy, xử lý hiệu ứng hình ảnh, âm thanh.

Phạm vi nghiên cứu:

- Xây dựng game hoàn thành ở mức độ cơ bản.

d. Phương pháp nghiên cứu:

- Nghiên cứu lý thuyết về lập trình game và thiết kế game để hiểu rõ các nguyên lý cơ bản.
- Sử dụng internet và các tài liệu tham khảo để học hỏi từ các tài liệu, hướng dẫn, và cộng đồng phát triển game Unity.
- Áp dụng kiến thức đã học vào việc xây dựng game theo các mục tiêu đề ra, thử nghiệm và điều chỉnh để đạt được sản phẩm hoàn thiện.

Chương 2: Cơ sở lý thuyết

I. Tổng quan về Unity Engine:

1.1. Unity là gì:

Unity là một “cross- platform game engine” tạm hiểu là công cụ phát triển game đa nền tảng được phát triển bởi Unity Technologies. Game engine này được sử dụng để phát triển game trên PC, consoles, thiết bị di động (Android, IOS) và trên websites. Unity3D là phần mềm làm games trực tiếp theo thời gian thực, mà không cần render, cho phép người design game có thể thiết kế InterfaceGraphic, map hay character,... từ một phần mềm thứ 2 (thường là các phần mềm thiết kế đồ họa chuyên nghiệp như 3Dsmax, Blender, Maya, XSL, Cinema4D, Cheetah3D, Modo, Autodesk FBX, LightWave...) sau đó chỉ việc import nó vào trong Unity với định dạng của tập tin là *FBX hay *.dae, *3DS, *.dxf và *.obj.

Chỉ với khoản tiền bỏ ra khá khiêm tốn (1.500 USD) là phiên bản pro đã nằm trong tay của chúng ta, dĩ nhiên tại Việt Nam số tiền này vẫn là quá lớn nhưng thật may là đã có phiên bản Unity Free. Tuy nhiên, nhiều tính năng quan trọng (Network) bị cắt giảm nhưng đó không phải là vấn đề quá lớn nếu muốn phát triển một tựa game tầm trung. Vào năm 2009, Unity nằm trong top 5 game engine tốt nhất cho việc sản xuất game với chỉ sau 4 năm phát triển. Unity đứng thứ 4, xếp sau Unreal Engine 3, Gamebryo Engine (được VTC Studic mua về phát triển SQUAD) và Cry Engine 2. Lượng tài liệu hướng dẫn Unity rất phong phú. Hơn thế nữa nó còn có sẵn một cộng đồng cực lớn với diễn đàn riêng. Bất cứ điều gì không hiểu chúng ta đều có thể thoải mái hỏi và nhận được câu trả lời nhanh chóng, tận tâm. Quá trình tạo địa hình cũng như truy xuất từ các phần mềm 3DSMax, Maya, Cinema4D... rất nhanh chóng. Sức mạnh và sự tiện lợi của Unity là vô cùng lớn.

Sức mạnh: Unity có thể tạo ra được nhiều loại game 2D, 3D đa dạng dễ sử dụng với người làm game chưa chuyên nghiệp, chất lượng cao, chạy hầu hết trên các hệ điều hành.

Sự tiện lợi: nếu chúng ta là một người chuyên dùng 3Dmax, hay Maya hoặc phần mềm mã nguồn mở Blender thì quả là thật tuyệt, chúng ta sẽ có một lợi thế lớn khi viết game trên Unity này, bởi công việc tạo các mô hình 2D, 3D sẽ trở lên dễ dàng hơn rất nhiều, việc kết hợp giữa người lập trình và

người thiết kế các mô hình sẽ nhanh và hiệu quả hơn. Trong Unity chúng ta có thể import trực tiếp các file mô hình đang thiết kế và sẽ thiết kế hoàn thiện tiếp nếu chưa xong trong khi đó công việc import chỉ diễn ra một lần. Không như việc phải dùng các công cụ khác để thực hiện viết game chúng ta sẽ phải xuất chúng ra một dạng nào đó và mỗi lần sửa lại phần mô hình chúng ta lại phải import lại, và như thế là quá mất thời gian trong việc tạo và chỉnh sửa các mô hình theo ý muốn. Ngoài ra Unity còn cho chúng ta trực tiếp tạo các mô hình nếu muốn. Việc đặt các thuộc tính vật lý trong Unity cũng cực kỳ dễ dàng và hỗ trợ sẵn nhiều chức năng.

1.2. Quá trình phát triển:

Phần lõi của Unity ban đầu được viết bởi Joachim Ante vào năm 2001. Sau đó công ty được hình thành vào năm 2005 và bắt đầu với phiên bản 1.0. Đến năm 2007, Unity được nâng lên phiên bản 2.0. Unity bắt đầu hỗ trợ iPhone vào năm 2008. Vào tháng 6/2010, Unity chính thức hỗ trợ Android và cho ra đời phiên bản 3.0 có hỗ trợ Android vào tháng 9/2010 và bây giờ là phiên bản Unity 5. Có thể thấy tốc độ phát triển của Unity khá nhanh.

Unity đã đoạt được nhiều giải lớn với những giải chính sau:

- Năm 2006, Unity đạt "Best Use of Mac OS X Graphics" tại Apple's WWDC. Đây là lần đầu tiên một công cụ phát triển game đạt được chất lượng do giải thưởng uy tín này đưa ra.
- Năm 2009, Unity Technologies có tên trong "Top 5 công ty game của năm" do Gamasutra tổ chức.
- Năm 2010, Unity đoạt giải Best Engine Finalist do Develop Magazine bình chọn, giải Technology Innovation Award của Wall Street Journal ở thể loại phần mềm.

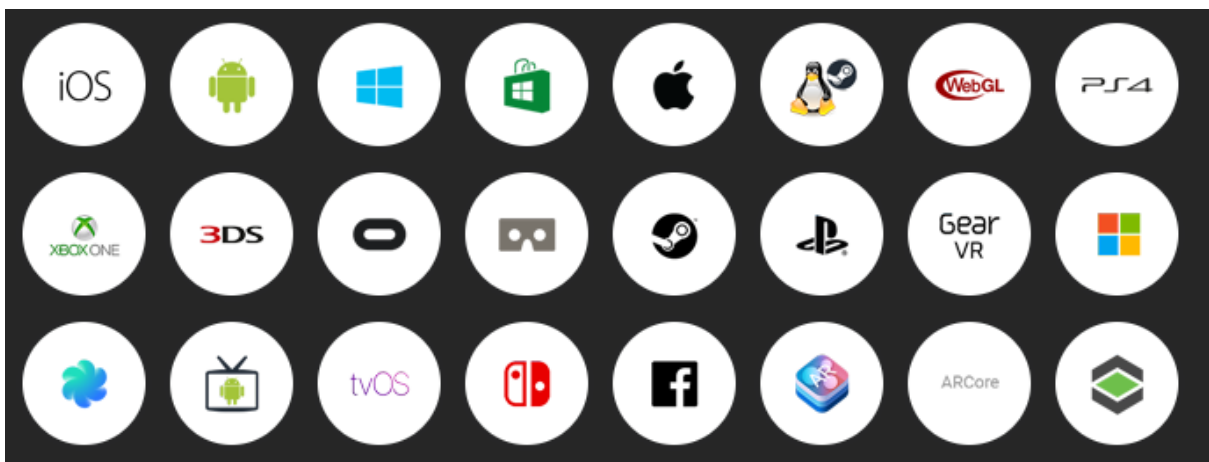
1.3. Một số thống kê về Unity:

Tính đến quý 3 năm được phát triển bởi Unity 2016 đã có 5 tỉ lượt download game và ứng dụng 2,4 tỉ thiết bị di động đã từng tải ít nhất 1 ứng dụng bởi unity. Trong top 1000 game Mobile miễn phí thì số lượng game tạo ra bởi Unity chiếm tới 34%. Số lượng người dùng (gamer) của Unity đạt tới con số 770 triệu, trong khi đó số người thường

xuyên sử dụng Twitter là 310 triệu người. (nguồn: thống kê từ internet)

1.4. Ưu điểm của Unity:

Chức năng cốt lõi đa dạng bao gồm: cung cấp công cụ dựng hình (kết xuất đồ họa) cho các hình ảnh 2D hoặc 3D, công cụ vật lý (tính toán và phát hiện va chạm), âm thanh, mã nguồn, hình ảnh động, trí tuệ nhân tạo, phân luồng, tạo dòng dữ liệu xử lý, quản lý bộ nhớ, dựng ảnh đồ thị và kết nối mạng. Nhờ có các engine mà công việc làm game trở nên ít tốn kém và đơn giản hơn.



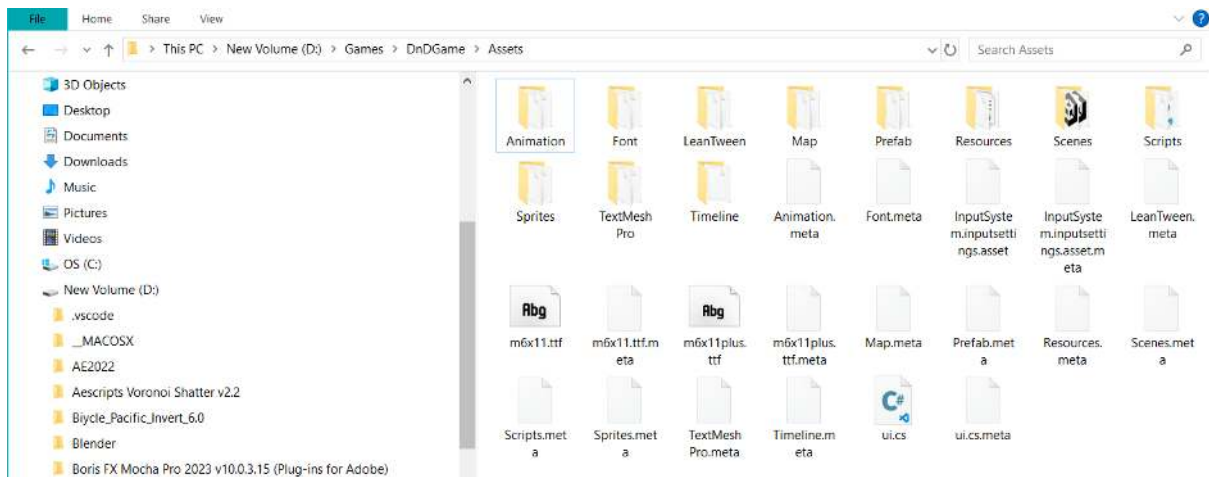
Hình 1:Unity Platform Support

Hỗ trợ đa nền tảng: Một trong các thế mạnh của Unity3D chính là khả năng hỗ trợ gần như toàn bộ các nền tảng hiện có bao gồm: PlayStation 3, Xbox 360, Wii U, iOS, Android, Windows, BlackBerry 10, OS X, Linux, trình duyệt Web và cả Flash. Nói cách khác, chỉ với một gói engine, các studio có thể làm game cho bất kỳ hệ điều hành nào và dễ dàng convert chúng sang những hệ điều hành khác nhau. Đồng thời, đây cũng là giải pháp cho các game online đa nền tảng có thể chơi đồng thời trên nhiều hệ điều hành, phần cứng khác nhau như Web, PC, Mobile, Tablet....

II. Các thành phần trong Unity:

2.1. Assets:

Assets là tài nguyên xây dựng nên một dự án trên Unity. Những tài nguyên có thể là hình ảnh, âm thanh, mô hình 2D, 3D, chất liệu (material), texture, ... hoặc cả một project hoàn chỉnh. Các asset do chính những nhà phát triển game tạo ra và có thể được download miễn phí hoặc trả phí trên Unity Asset Store. Đây là một trong những tính

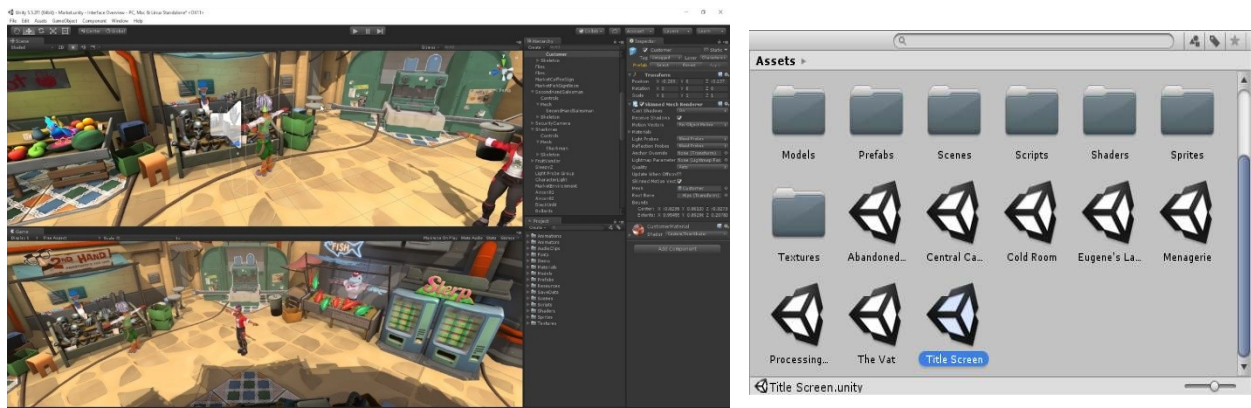


năng rất hay của Unity. Các asset này sẽ giúp giảm thiểu rất nhiều thời gian cho việc thiết kế và lập trình game.

Hình 2.1: Asset trong Unity

2.2. Scenes:

Là một cảnh chơi (hoặc một phân đoạn) là những màn chơi riêng biệt, một khu vực trong game hoặc thành phần có trong nội dung của trò chơi (các menu). Các thành phần này được gọi là Scene. Bằng cách tạo ra nhiều Scenes,

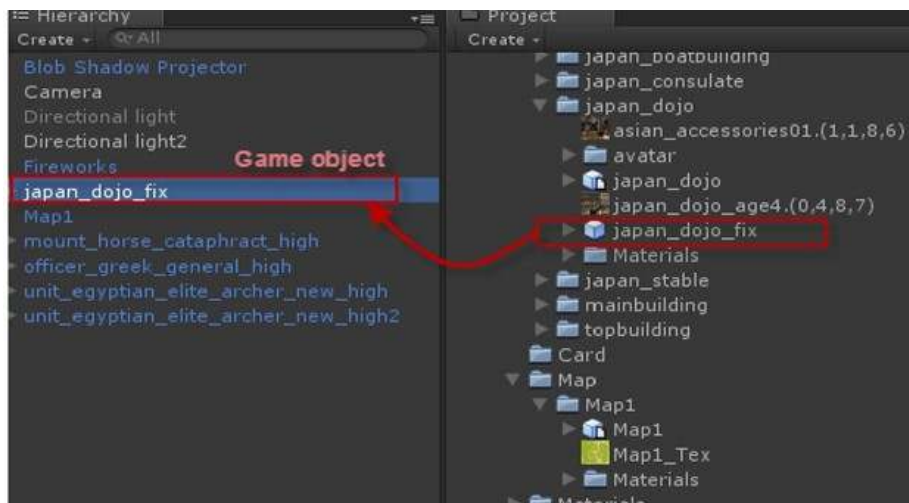


chúng ta có thể phân phối thời gian và tối ưu tài nguyên, kiểm tra các phân đoạn trong game một cách độc lập.

Hình 2.2: Các Scene trong Unity

2.3. Game Objects:

Khi Asset được sử dụng trong các Scene, Unity định nghĩa đó là Game Object. Đây là một thuật ngữ thông dụng, đặc biệt trong mảng lập trình. Tất cả các Game Object đều chứa ít nhất một thành phần cơ bản là Transform, lưu trữ thông tin về vị trí, góc xoay và tỉ lệ của Game Object. Thành



phần Transform có thể được tùy biến và chỉnh sửa trong quá trình lập trình.

Hình 2.3: Kéo tài nguyên vào sử dụng

2.4. Components:

Components là các thành phần trong game, bổ sung tính năng cho các Game Object. Mỗi Component có chức năng riêng biệt. Đa phần các Component phụ thuộc vào Transform, vì nó lưu trữ các thông số cơ bản của Game



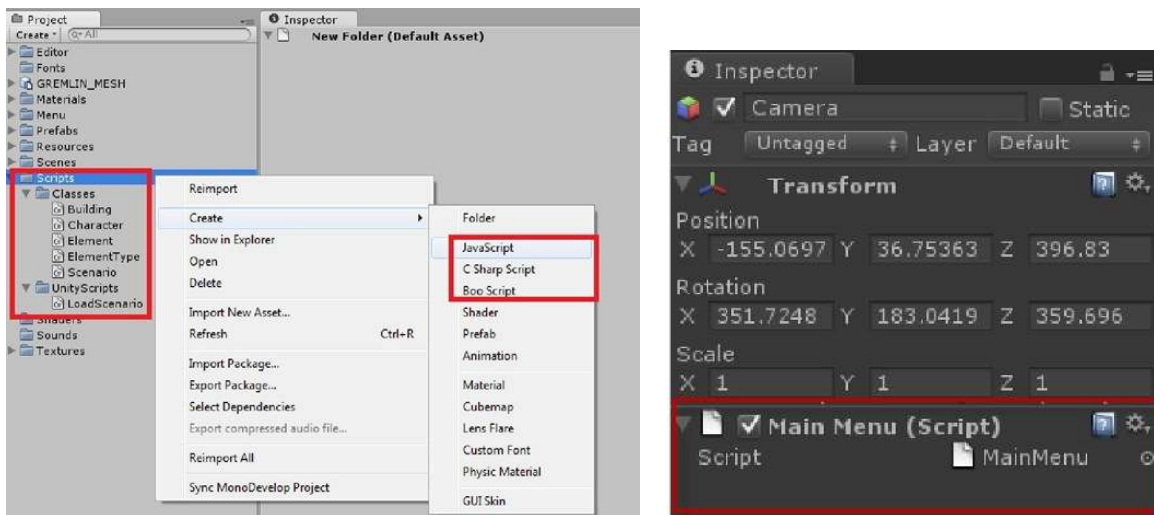
Object. Bản chất của Game Object là không có gì cả các đặc tính và khả năng của Game Object nằm hoàn toàn trong các Component. Do đó chúng ta có thể xây dựng nên bất kỳ Game Object nào trong game mà chúng ta có thể tưởng tượng được.

Hình 2.4: Các thành phần trong Camera

2.5. Scripts:

Scripts được Unity xem như một Component. Đây là thành phần thiết yếu trong quá trình phát triển game. Bất kỳ một game nào, dù đơn giản nhất đều cần đến Scripts để tương tác với các thao tác của người chơi, hoặc quản lý các sự kiện để thay đổi chiều hướng của game tương ứng với

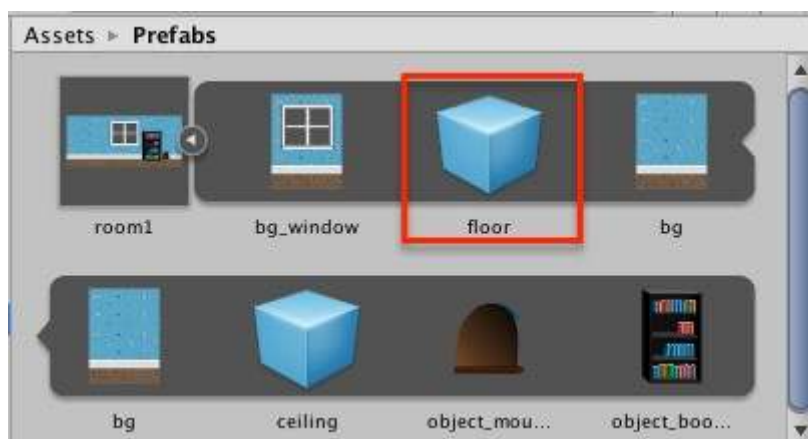
kịch bản game. Unity cung cấp cho lập trình viên khả năng viết Script bằng các ngôn ngữ: JavaScript, C#. Unity không đòi hỏi lập trình viên phải học cách lập trình trong Unity, nhưng trong nhiều tình huống, chúng ta cần sử dụng Script trong mỗi phần của kịch bản game. Để viết Script, chúng ta có thể làm việc với một trình biên tập Script độc lập của Unity; hoặc làm việc trên Mono Developer được tích hợp vào Unity trong những phiên bản gần đây. Mono Developer là một IDE khá tốt, cung cấp nhiều chức năng tương tự Visual Studio. Mã nguồn viết trên Mono Developer sẽ được cập nhật và lưu trữ trong dự án trên Unity.



Hình 2.5: Cách tạo Script và gắn vào GameObject

2.6. Prefabs:

Prefabs thực chất là Game Object được lưu trữ lại để tái sử dụng. Các Game Object được nhân bản từ một prefab sẽ giống nhau hoàn toàn, ngoại trừ thành phần Transform



để phân biệt và quản lý được tốt hơn. Để tạo ra một prefab, ta đơn giản chỉ cần kéo một Game Object vào cửa sổ Project.

Hình 2.6: 1 số đối tượng trong Prefabs

2.7. Material và Shader:

Shader là đoạn script quy định cách thức render của chất liệu trên bề mặt vật thể. Material sử dụng shader để làm chất liệu cho mô hình. Giữa materials và shaders có mối liên hệ với nhau. Shaders quy định các thuộc tính cần để shader làm việc. Còn material cho phép gán hình ảnh vào các thuộc tính đó từ Asset.

2.8. UnityAPI:

UnityAPI chứa rất nhiều lớp hỗ trợ lập trình game, trong đó có một số lớp quan trọng như :

- MonoBehaviour: tất cả các script muốn gắn vào một đối tượng game bắt buộc phải kế thừa từ lớp này.
- GameObject: lớp cha của tất cả các thực thể trong scene.
- Component: lớp cha của tất cả các thành phần có thể gắn vào đối tượng.
- Transform: giúp thay đổi vị trí, xoay, biến đổi tỉ lệ mô hình. Input: hỗ trợ lập trình với chuột, cảm ứng đa điểm, cảm biến gia tốc.
- Camera: giúp lập trình camera.
- Light: giúp tạo ánh sáng trong game.
- Projector: giúp chiếu texture lên bề mặt vật thể.
- ParticleEmitter: hỗ trợ tạo các hiệu ứng particle đẹp mắt.
- Audio: hỗ trợ lập trình với âm thanh.
- Animation: chạy chuyển động của mô hình nhân vật.
- Rigidbody: giúp tạo hiệu ứng vật lý liên quan đến trọng lực như bóng nảy, lăn, ...
- CharacterController: giúp điều khiển nhân vật di chuyển theo độ cao địa hình.
- Collider: hỗ trợ lập trình va chạm giữa các vật thể.
- GUI: giúp lập trình giao diện người dùng trên Unity.

2.9. Không gian 3D và vector trong không gian:

Nhắc về hệ tọa độ không gian 3 chiều chắc hẳn chúng ta đều nhớ môn hình học không gian hoặc đã từng sử dụng

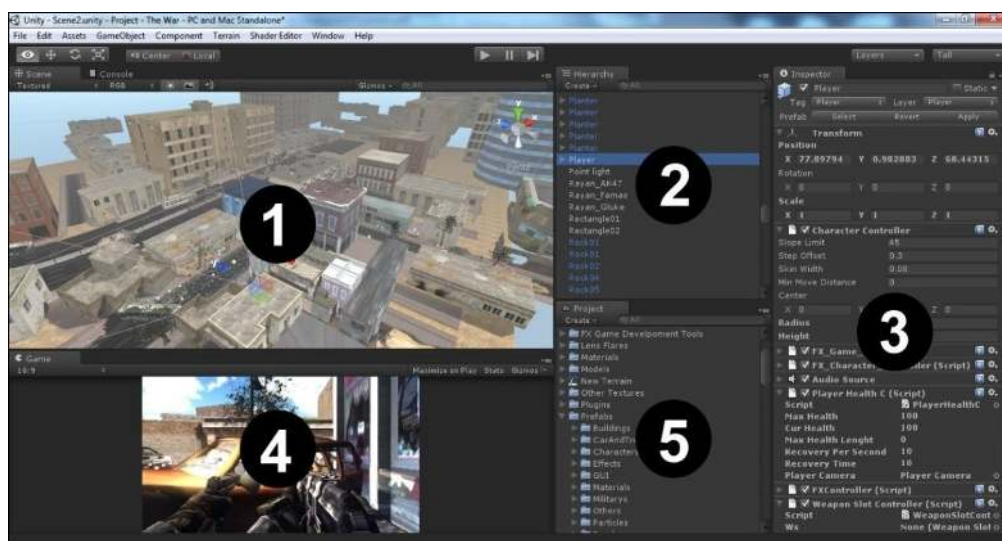
một công cụ dựng hình 3D nào đó. Hệ tọa độ không gian 3 chiều bao gồm 3 trục X, Y, Z hay hiểu đơn giản là chiều ngang (X) chiều cao (Y) và chiều sâu (Z). Chúng được kí hiệu theo cú pháp: (X, Y, Z).

Trong không gian 3 chiều, tồn tại 1 điểm gọi là Origin hoặc Word Zero. Đây là điểm có tọa độ (0,0,0). Tất cả tọa độ của các đối tượng tồn tại trong không gian đều có mối liên hệ với điểm này, cách tính này là theo Word Space. Tuy nhiên, để việc tính toán đơn giản, người ta sử dụng thêm một khái niệm gọi là Local Space nhằm xác định tọa độ của đối tượng này so với đối tượng khác. Mối quan hệ này gọi là parent – child.

III. Sơ lược về giao diện Unity:

3.1. Giao diện:

Giao diện Unity, giống như nhiều môi trường làm việc khác, layout có thể tùy chỉnh. Layout của Unity bao gồm nhiều tab khác nhau và có thể bật tắt.



Hình 2.7: Giao diện Unity

3.2. Cửa sổ Scene và Hierarchy:

Cửa sổ scene là nơi mà chúng ta sẽ xây dựng các thực thể, đối tượng của dự án vào đó. Cửa sổ cung cấp góc nhìn phối cảnh (Perspective (góc nhìn 3D), chúng ta có thể chuyển qua các góc nhìn khác như từ trên xuống hoặc từ dưới lên (Top Down), từ trái sang phải hoặc phải sang trái

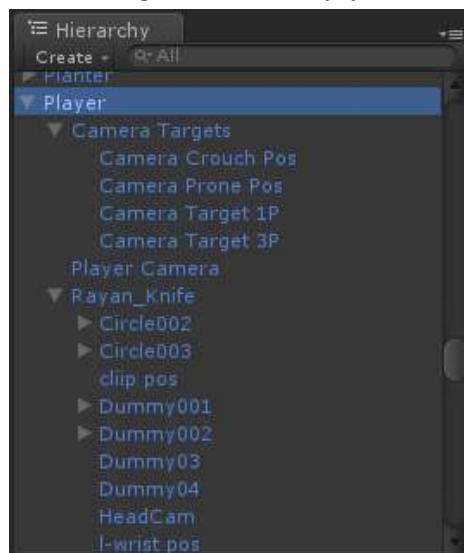


(Side On), từ trước ra sau hoặc sau đến trước (Front On). Cửa sổ này sẽ kết hình xuất đầy đủ những hình ảnh trong thế giới của trò chơi mà chúng ta tạo ra dưới dạng một vùng biên tập mà chúng ta có thể biên tập, chỉnh sửa trực tiếp thế giới đó.

Khi kéo thả Asset vào cửa sổ Scene, Assets sẽ trở thành Game Object. Cửa sổ Scene được ràng buộc cùng với cửa sổ Hierarchy, cửa sổ Hierarchy liệt kê danh sách các Game Object có trong Scene và được sắp xếp theo thứ tự chữ cái từ A-Z.

Cửa sổ Scene còn đi kèm với 4 bốn nút chức năng hữu ích được hiển thị dưới dạng hình ảnh như trên. Chúng có thể được lựa chọn thông qua các phím tắt Q, W, E và R. Những nút này có các chức năng như sau:

- Công cụ bàn tay (Q): Công cụ này cho phép chúng ta di chuyển đến một khu vực nào đó trong Scene bằng thao tác kéo thả thuộc trái.
- Công cụ di chuyển (W): Công cụ này cho phép chúng ta chọn một đối tượng trong cảnh và thực hiện thao tác di chuyển, thay đổi vị trí của đối tượng đó. Khi chọn, tại vị trí của đối tượng sẽ hiển thị các trục và mặt phẳng gắn liền với đối tượng cho phép chúng ta di chuyển đối tượng trượt theo các trục, mặt phẳng hoặc di chuyển một cách tùy ý.
- Công cụ xoay (E): Công cụ này có đặc điểm và cách sử dụng giống với công cụ di chuyển, tuy nhiên thay vì để di chuyển vị trí của đối tượng thì công cụ này giúp chúng ta xoay đối tượng xoay quanh trục hay tâm của đối tượng.
- Công cụ điều chỉnh tỉ lệ (R): Cũng tương tự như công cụ di chuyển và xoay, công cụ này cho phép chúng ta tùy chỉnh kích thước, tỉ lệ của đối tượng một cách tùy ý.

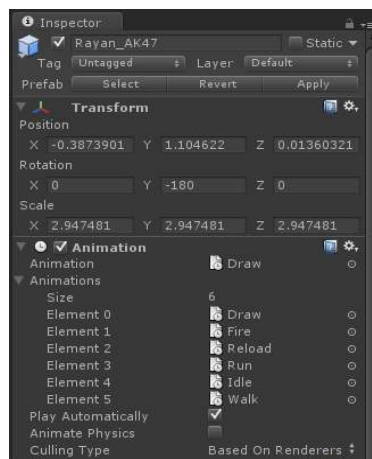


Hình 2.8: Cửa sổ Hierarchy

3.3. Cửa sổ Inspector:

Cửa sổ Inspector có thể được xem như một công cụ cho phép chúng ta tùy chỉnh các thiết đặt, các thành phần của Game Object hoặc Assets đang được chọn. Cửa sổ này sẽ hiển thị đầy đủ các Components của đối tượng mà chúng ta chọn. Nó cho phép chúng ta điều chỉnh các biến của Components dưới các hình thức như: Textbox, Slider, Button, Drop-down Menu...

Ngoài việc hiển thị các Component của đối tượng được chọn, cửa sổ Inspector còn hiển thị các thiết đặt

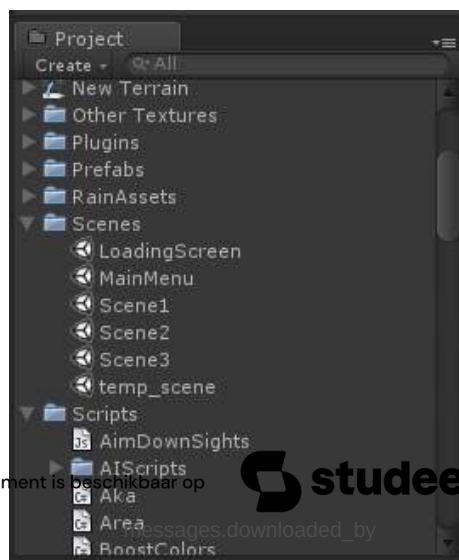


chung của hệ thống hay của trò chơi khi ta chọn chúng từ menu Edit.

Hình 2.9: Cửa sổ Inspector

3.4. Cửa sổ Project:

Cửa sổ Project là cửa sổ cho phép chúng ta nhìn thấy trực tiếp nội dung của thư mục Assets của dự án. Mỗi dự án Unity đều được chứa trong một thư mục cha. Trong đó có 3 thư mục con là Assets, Library và Temp (chỉ có khi Unity đang chạy). Đặt tất cả các Assets vào thư mục Assets có nghĩa là ngay lập tức chúng ta sẽ thấy chúng xuất



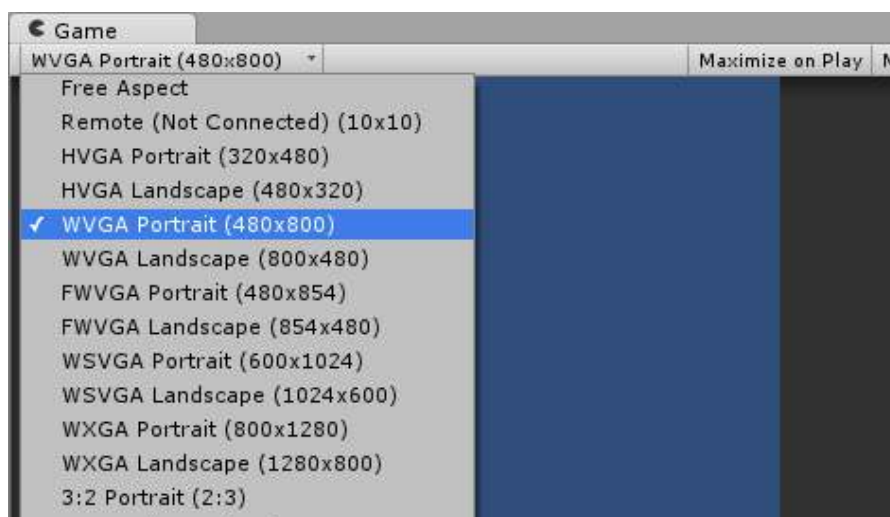
hiện trong cửa sổ Project. Ngoài ra, khi thay đổi vị trí của Assets trong thư mục Assets hay lưu tập tin lại từ một chương trình ứng dụng thứ 3 nào khác (ví dụ như Photoshop), sẽ làm cho Unity nhập lại (Re-Import) Assets, phản ánh sự thay đổi này ngay lập tức trong cửa sổ Project và Scene có sử dụng Assets vừa được thay đổi.

Hình 3: Cửa sổ Project

3.5. Cửa sổ Game:

Cửa sổ Game sẽ được gọi khi chúng ta nhấn vào nút Play (là một hành động thực hiện test trò chơi). Cửa sổ này cho phép chúng ta tùy chọn về thiết đặt tỉ lệ màn hình, nó phản ánh phạm vi trong Scene mà người chơi có thể thấy được với mỗi tỉ lệ màn hình tương ứng, ví dụ như với mỗi tỉ lệ màn hình 4:3, 16:9 thì người chơi sẽ có một phạm vi nhìn thấy khác nhau.

Sau khi nhấn vào nút Play, chúng ta sẽ ở chế độ Testing, lúc này mọi thay đổi về các thuộc tính, Components, ... của đối tượng sẽ chỉ là tạm thời. Tức là chúng sẽ trở về



như ban đầu (trước khi nhấn nút Play) sau khi kết thúc chế độ Testing.

Hình 3.1: Các loại hình ảnh trong Game

IV. Tìm hiểu về công cụ hỗ trợ :

4.1. Giới thiệu:

Visual Studio là (IDE – Integrated Development Environment) một bộ công cụ phát triển phần mềm do Microsoft phát triển. Visual Studio cũng là một phần mềm được sử dụng bởi các lập trình viên để xây dựng nên các sản phẩm phần mềm.



4.2. Lý do nên dùng Visual Studio:

- Hỗ trợ lập trình trên nhiều ngôn ngữ như C/C+, C#, F#, Visual Basic, HTML, CSS, JavaScript. Phiên bản Visual Studio 2015 có hỗ trợ ngôn ngữ Python
 - Visual Studio là một công cụ hỗ trợ việc Debug một cách mạnh mẽ, dễ dàng nhất (Break Point, xem giá trị của biến trong quá trình chạy, hỗ trợ debug từng câu lệnh).
 - Giao diện Visual Studio rất dễ sử dụng đối với người mới bắt đầu.
- Visual Studio hỗ trợ phát triển ứng dụng desktop MFC, Windows Form, Universal App, ứng dụng mobile Windows Phone 8/8.1, Windows 10, Android (Xamarin), iOS và phát triển website Web Form, ASP.NET MVC và phát triển Microsoft Office.
- Visual Studio hỗ trợ kéo thả để xây dựng ứng dụng một cách chuyên nghiệp. giúp các bạn mới bắt đầu có thể tiếp cận nhanh hơn.
 - Visual Studio cho phép chúng ta tích hợp những extension từ bên ngoài như Resharper (hỗ trợ quản lý và viết mã nhanh cho các ngôn ngữ thuộc Net), hay việc cài đặt thư viện nhanh chóng thông qua Nuget.
 - Visual Studio được sử dụng đông đảo bởi lập trình viên trên toàn thế giới.

4.3. Tổng quan về C#:

C# là một ngôn ngữ lập trình theo phương pháp lập trình hướng đối tượng và lập trình trực quan được phát triển bởi Microsoft.

C# ra đời năm 2000, được thiết kế chủ yếu bởi Anders Hejlsberg – kiến trúc sư phần mềm nổi tiếng với các sản phẩm Turbo Pascal, Delphi...

Được xây dựng trên nền tảng của hai ngôn ngữ lập trình mạnh nhất đó là C++ và Java. Do đó C# được miêu tả là ngôn ngữ có sự cân bằng giữa C++, Pascal, Delphi và

Java. C# với sự hỗ trợ mạnh mẽ của .NET Framework giúp cho việc tạo một ứng dụng Windows Forms hay WPF (Windows Presentation Foundation).

4.4. Các đặc điểm của C#:

- Là một ngôn ngữ hướng đối tượng, trực quan.
- Tám mươi từ khóa và hơn mười kiểu dữ liệu được dựng sẵn trở nên rất dễ.
- Cung cấp những đặc tính hướng thành phần (component - oriented) như là Property, Event.
- C# không khuyến khích sử dụng con trỏ như C++.
- C# có bộ Garbage Collector (thu gom rác) sẽ tự động thu gom vùng nhớ khi không còn sử dụng nữa.
- C# đã loại bỏ đa kế thừa trong C++ mà thay vào đó C# sẽ hỗ trợ thực thi giao diện interface.

Nguyên nhân lựa chọn C# là vì:

- C# là một ngôn ngữ đơn giản.
- C# là một ngôn ngữ hiện đại.
- C# là một ngôn ngữ mạnh mẽ và mềm dẻo.
- C# là một ngôn ngữ ít từ khóa.
- C# là một ngôn ngữ phổ biến.
- C# mang sức mạnh của nhiều ngôn ngữ (C++, Java....).

4.5. Ưu điểm và nhược điểm của C#:

Ưu điểm:

- Dễ học hơn 6 tháng.
- Dễ đọc mã hơn C++.
- Phát triển nhanh hơn và có khả năng ít lỗi hơn so với C++ hoặc Java (có các loại không dấu, có ref / out, có thể tạo các loại giá trị của riêng mình, có những thứ hữu ích khác mà Java bỏ qua có nghĩa là bạn ít nhảy qua vòng có nghĩa là độ phức tạp mà ít cần thiết hơn).
- Tất cả mọi thứ được truyền bằng tham chiếu ngoại trừ các loại giá trị theo mặc định.
- Công cụ thu gom rác dọn dẹp đồ vật một khi chúng không còn được sử dụng, vì vậy bạn không phải tự mình theo dõi mọi thứ.
- Chương trình được biên dịch thành các nhị phân riêng được tối ưu hóa cho nền tảng khi nó được chạy (và nó chạy chậm hơn một chút so với C++).
- Dễ dàng tạo nhiều chủ đề Có nhiều phương tiện tạm dừng các luồng để chờ tin hiệu và như vậy.
- lock (someObject) {code}, giống như Java được đồng bộ hóa nhưng có thể được sử dụng ở bất cứ đâu và yêu cầu một đối tượng.
- P/ Gọi để sử dụng hơn JNI rất nhiều - nhưng có lẽ chúng tôi sẽ không sử dụng nó vì chúng tôi muốn trở thành đa nền tảng.

- Có một IDE miễn phí tuyệt vời (visual c # express) - nhưng chỉ trên windows (xem khuyết điểm).
- Có thể tạo các loại giá trị, theo mặc định là truyền theo giá trị thay vì truyền bằng tham chiếu, bằng cách tạo cấu trúc thay vì một lớp.
- Có các từ khóa ref và out cho phép bạn chuyển tham chiếu đến tham số cho hàm, với ý nghĩa là tham số đó phải được gán bởi hàm trước khi trả về. Về cơ bản, ref / out cho phép hàm sửa đổi biến được truyền dưới dạng tham số, giống như chuyển tham chiếu trong C.
- Đa nền tảng với mono và / nhưng mono vẫn đang được cải thiện. Có số nguyên không dấu (c ++, java không).
- Nếu chương trình gặp sự cố, nó sẽ bật lên một hộp thoại cho người dùng biết mã ở đâu và tại sao (trên windows) hoặc ghi dấu vết ngăn xếp vào bản điều khiển (với mono nếu chạy với --debug).
- Các chương trình được biên dịch thành các tập exe và không cần phải biên dịch lại cho các HĐH khác - mono có thể chạy trực tiếp exes NET.

Nhược điểm:

- Sử dụng nhiều bộ nhớ hơn C++.
- Trình thu gom rác sử dụng chu kỳ và bộ nhớ CPU - (nhưng hầu như không có, ít nhất là thời gian CPU).
- Một số thứ không hoạt động trong các chức năng gọi đơn trong một số cửa sổ sẽ không tồn tại trên linux, việc triển khai các biểu mẫu đơn của windows cần có cách giải quyết trong mã (nhưng có lẽ chúng ta sẽ không sử dụng chúng).
- Mono chưa hoàn hảo.
- Phải sử dụng net 2.0 cho thuốc generic (trong số những thứ khác), 1.1 không có chúng. Không thể sử dụng bất cứ thứ gì mới hơn 2.0 vì hỗ trợ trả 3.0 trở lên cho windows 98, ME và có thể 2000.
- Không thể khai báo các chức năng được đồng bộ hóa.
- Không có IDE tốt trên các cửa sổ không. Eclipse có plugin ac # nhưng nó có thể không hỗ trợ hầu hết các tính năng của Eclipse.
- Không có con trỏ, nhưng chúng được thay thế bởi các tham số ref và out, vì vậy chúng không cần nhiều.
- Phương thức gọi thông qua các đại diện chậm hơn đáng kể so với cách gọi phương thức bình thường (hoặc con trỏ hàm trong C++, có lẽ).
- Yêu cầu (phiên bản chính xác) của NET framework phải được cài đặt để chạy chương trình, đây là một bản tải xuống vài giờ khi quay số.

4.6. Phần mềm Aseprite:

Aseprite là một phần mềm chuyên dụng để tạo và chỉnh sửa đồ họa pixel, hoạt hình sprite, và các tài nguyên đồ họa cho trò chơi điện tử. Nó được yêu thích bởi cộng đồng nghệ sĩ đồ họa pixel và nhà phát triển game indie nhờ vào giao diện thân thiện và các tính năng mạnh mẽ.

Tính năng chính:

1. Chỉnh sửa đồ họa pixel:
 - Aseprite cung cấp các công cụ chỉnh sửa đồ họa pixel như bút vẽ, tẩy, làm mờ, và các công cụ khác để tạo và chỉnh sửa chi tiết từng pixel một.
 - Công cụ quản lý lớp (layer) giúp người dùng dễ dàng làm việc với các phần khác nhau của hình ảnh mà không ảnh hưởng đến các phần còn lại.
2. Tạo hoạt hình (Animation):
 - Công cụ timeline mạnh mẽ cho phép người dùng tạo và quản lý các khung hình (frames) của một hoạt hình.
 - Các tính năng hỗ trợ như onion skinning giúp dễ dàng xem các khung hình trước và sau, tạo ra các chuyển động mượt mà hơn.
3. Hỗ trợ đa nền tảng:
 - Aseprite có thể chạy trên nhiều hệ điều hành khác nhau như Windows, macOS và Linux.
4. Xuất và nhập định dạng:
 - Hỗ trợ xuất và nhập nhiều định dạng hình ảnh khác nhau như PNG, GIF, BMP, và các định dạng sprite sheet, giúp dễ dàng tích hợp với các công cụ phát triển game khác.
5. Script và mở rộng:
 - Hỗ trợ scripting bằng ngôn ngữ Lua, cho phép người dùng tạo các công cụ tùy chỉnh và tự động hóa các tác vụ lặp lại.
6. Giao diện người dùng:
 - Giao diện đơn giản và thân thiện với người dùng, giúp dễ dàng truy cập các công cụ và tính năng.
 - Các phím tắt và khả năng tùy chỉnh giao diện để phù hợp với phong cách làm việc của từng cá nhân.

Chương 3: Mô tả thiết kế game

I. Thiết kế:

1. Cốt truyện:

- Thể loại:
 - RPG.
 - Top down.
 - Đối tượng người chơi: 10+ tuổi.
- Ý tưởng:

Trong một chuyến thám hiểm Dungeon, tổ đội của Laios vô tình rơi vào một cánh cửa bí mật dẫn tới một tầng Dungeon bí ẩn. Ở đó họ đã đối mặt với một con rồng và gần như bị tiêu diệt. Cho đến khi cô em gái Falin niệm một ma pháp dịch chuyển toàn bộ tổ đội ra khỏi Dungeon trừ bản

thân. Laios sau đó tập hợp mọi người tiếp tục tiến sâu vào hầm ngục với mục đích chính là tìm kiếm em gái mình.

Người chơi sẽ hóa thân thành nhân vật Laios với nhiệm vụ đi chiến đấu với các con quái vật qua từng màn chơi để tiến đến giết boss cuối và giành chiến thắng trò chơi.

1.2. Game Setting:

a. Nhân vật:

- Hiệp sĩ: Laios

Backstory: Một nhân loại, đội trưởng đội thám hiểm, khao khát khám phá hầm ngục và cả những con quái vật.

Tính cách: chính trực, thành thật, rất quan tâm đến đồng đội và có hứng thú đặc biệt với ma vật.

Sử dụng kiếm, chỉ số cân bằng, sử dụng một số ma pháp thánh hệ.



Hình 3.2: Nhân vật Laios

- Tu sĩ: Falin

Backstory: Một nhân loại, thân với Laois từ nhỏ và là bạn học của Falin, cô hy sinh để nhiệm phép dịch chuyển tổ đội khỏi hầm ngục bí ẩn.

Tính cách: Hòa nhã, yêu quý động vật và được mọi người yêu quý.



Hình 3.2: Nhân vật Falin

- Pháp sư: Marcille

Backstory: Một Elf đến từ phương Bắc, cô ấy có xuất thân khá cao quý, quen biết Falin trong trường học ma pháp.

Tính cách: Hơi nóng nảy, bề ngoài khó gần nhưng thực ra cũng rất quan tâm đến mọi người, có niềm say mê lớn đối với nghiên cứu ma thuật.

Sử dụng gậy phép, chỉ số ma thuật cao nhưng phòng thủ kém, dụng ma pháp hệ hỏa.



Hình 3.4: Nhân vật Marcille

- Đạo chích: Chilchuck

Backstory: Một bán nhân đến từ phía Tây, anh ta giấu xuất thân của mình.

Tính cách: Vui vẻ, dễ gần nhưng nghiêm túc khi làm việc.



Hình 3.5: Nhân vật ChilChuck

- Người hỗ trợ: Katherine

Backstory: Katherine là nhân viên của Hiệp Hội Nhà Mạo Hiểm, hỗ trợ và hướng dẫn các nhà mạo hiểm tại các thành phố lớn.

Tính cách: tận tụy, thân thiện, chuyên nghiệp và luôn sẵn sàng giúp đỡ.

Katherine không tham gia chiến đấu và không sử dụng vũ khí, cô hỗ trợ người chơi



qua việc cung cấp nhiệm vụ và thông tin.

Hình 3.6: Nhân vật Katherine

b. Kẻ địch:

- Slime (Quái nhầy): dùng chất nhờn tiếp cận, gây sát thương lên người chơi.



Hình 3.7: Slime

- WildPig (Lợn Rừng): tấn công người chơi bằng khả năng phóng to cặp sừng đây chết chóc, đòn húc.



Hình 3.8: WildPig

- RedDragon (Rồng Đỏ): là 1 loại boss mà người chơi phải đối đầu khi vào game, được cho là con boss siêu khỏe, có thể gây sát thương diện rộng, khắc lửa lên người chơi.



Hình 3.9: Boss RedDragon

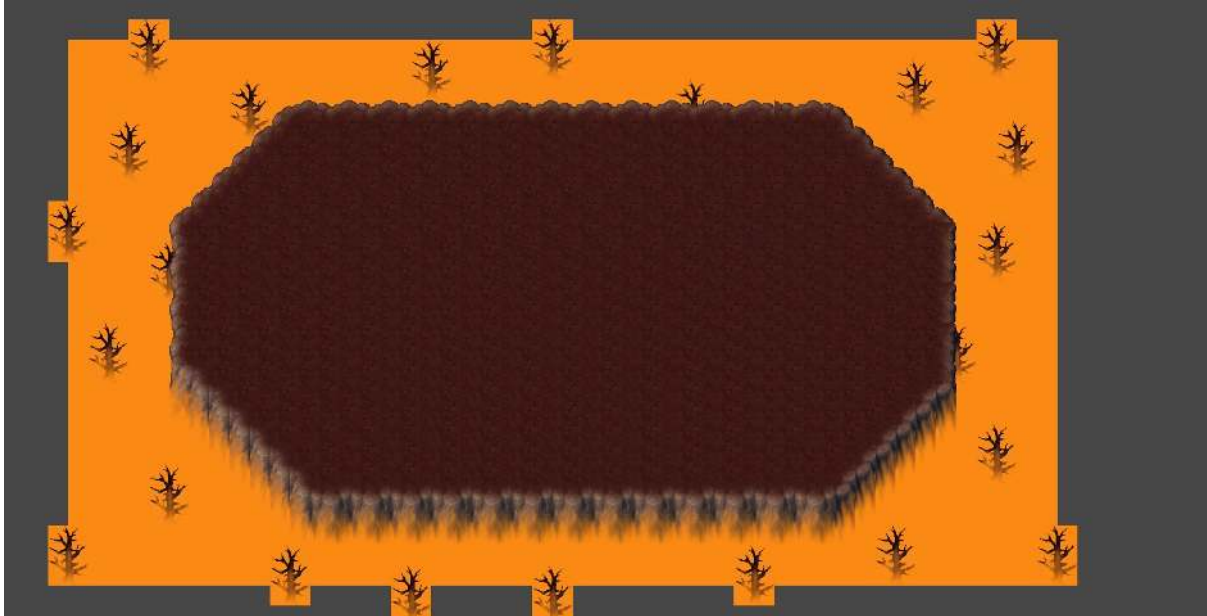
- Satan (Quỷ địa ngục a.k.a Lucifer Morningstar): Là con boss cuối cùng mà người chơi phải đương đầu, lúc đầu có hình dạng là slime đỏ, sau khi chết lần 1 sẽ hóa thân thành chúa quỷ có khả năng tấn công, phòng thủ toàn diện.



Hình 4: Boss Satan

c. Bản đồ:

- Map núi lửa: Là tầng sâu nhất trong dungeon, lấy bối cảnh vùng đất cằn cỗi bao xung quanh bởi dòng dung nham nhóng chảy. Nơi mà boss RedDragon trú ngụ, người chơi sẽ phải chiến đấu với boss rồng tại đây.



Hình 4.1: Map lửa

- Map thành phố:

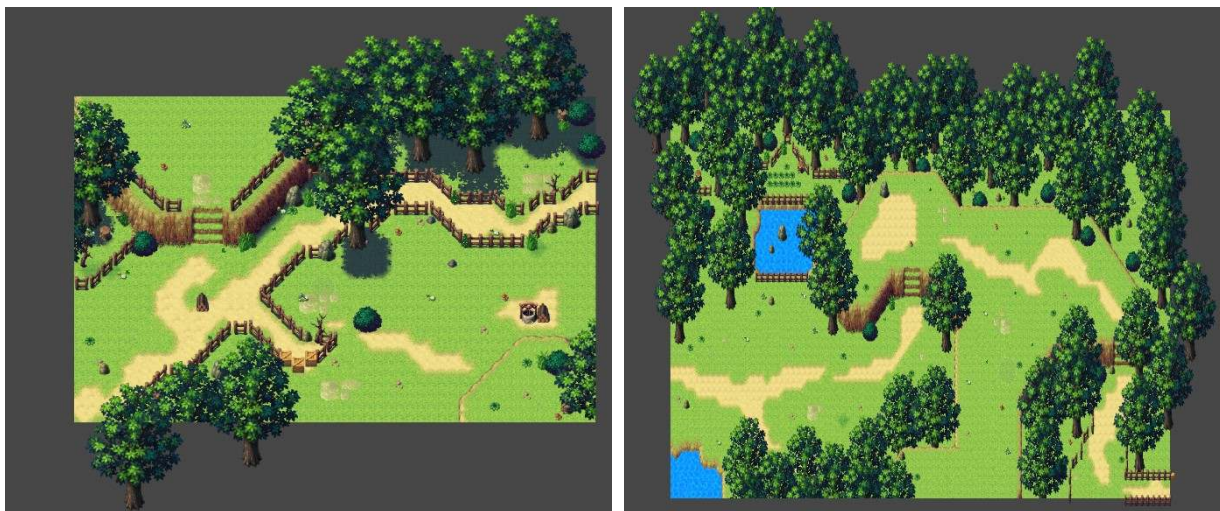
Lấy ý tưởng từ Zenonia, là 1 thành phố nhỏ nơi sinh sống của các dân làng, tại đây người chơi sẽ tương tác với các NPCs, là nơi dừng chân nghỉ ngơi của các mạo hiểm giả trước khi đến dungeon.

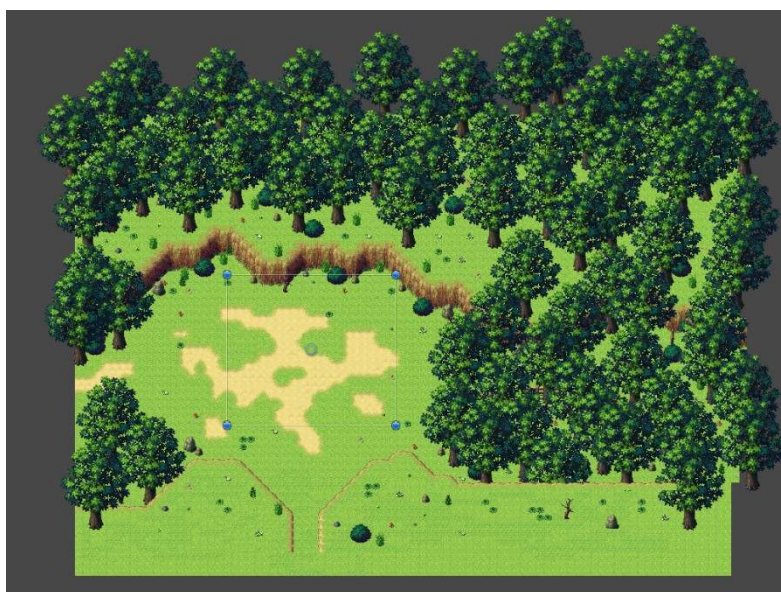


Hình 4.2: Map thành phố

- Map rừng:

Là dungeon tầng đầu tiên, địa hình chính bao gồm nhiều con đường mòn màu vàng chạy xuyên qua khu vực, tạo thành các lối đi khác nhau. Phía trên bên trái của bản đồ có một bậc thang đất đá, dẫn lên một khu vực cao hơn. Bản đồ được bao phủ bởi cỏ xanh tươi và rải rác một số điểm có hoa nhỏ mọc lên, tạo nên một khung cảnh tươi mát và sinh động. Nhiều cây lớn với tán lá xanh rậm rạp được bố trí khắp nơi, tạo bóng râm cho các khu vực xung quanh. Tại đây người chơi sẽ phải chiến đấu với quái nhầy (slime) và lượn rừng qua 2 map sau đó sẽ tới map đánh boss cuối là Satan.





Hình 4.3: Map rừng

II. Cơ chế và Tiến trình chơi:

2.1. Tiến trình trò chơi:

- Trước khi tiến hành chơi người chơi sẽ được chiến đấu với Boss mạnh nhất của game và bị tiêu diệt, sau đó người chơi sẽ được xem 1 đoạn scene ngắn trước khi bắt đầu các màn chơi chính.
- Người chơi sẽ được tương tác với các NPC trước khi bắt đầu nhiệm vụ tiêu diệt các con quái có trong game.
- Người chơi sẽ phải khéo léo di chuyển nhân vật và tiêu diệt những con quái từ các hầm ngục khác nhau để tiến đến hầm ngục sâu nhất và triệt hạ Boss cuối của game.
- Nhân vật của người chơi sẽ được trang bị vũ khí (kiếm) đi cùng bộ kỹ năng, người chơi sẽ phải tiêu diệt hết các con quái thì mới có thể tiến đến màn tiếp theo.
- Số lượng quái trong mỗi màn là rất nhiều, người chơi sẽ phải cố gắng “giữ mạng” để sống sót vượt qua các màn tiếp theo.
- Trò chơi kết thúc khi người chơi bị quái tiêu diệt hoặc tiêu diệt thành công Boss cuối của game.
- Từ giao diện chính người chơi có những lựa chọn sau:
 - Play – Bắt đầu trò chơi.
 - Option – Cài đặt trò chơi.
 - Tutorial – Hướng dẫn chơi.
 - Exit – Thoát trò chơi.

2.2. Cơ chế chơi:

- Thám hiểm, di chuyển lên, xuống, trái phải, lướt.
- Các đối tượng trên bản đồ: Quái vật, Người chơi.
- Tương tác: Nói chuyện với NPC.

Cơ chế chiến đấu:

- Di chuyển (4 hướng), tấn công theo hướng con trỏ chuột.
- Kẻ địch ko có thanh máu cụ thể mà dựa trên số lần đánh, đánh trúng kẻ địch số lần cụ thể để tiêu diệt kẻ địch. Có thể giảm số lượt đánh lên địch khi sử dụng kỹ năng của nhân vật.
- Kết thúc chiến đấu: hạ gục toàn bộ quái vật sẽ thắng trò chơi.
- Nếu người chơi bị hạ, trở lại địa điểm.

Màn chơi:

Prologue:

Tóm tắt: tổ đội xuất hiện ở tầng Dungeon bí ẩn, cố gắng đánh bại con rồng nhưng bất thành, một đồng đội hi sinh để dịch chuyển mọi người lên mặt đất.

Địa điểm: tầng sâu nhất: Cổng Dungeon.

Mục tiêu: Cố đánh bại con rồng (RedDragon) nhưng thất bại, sau khi thua boss người chơi sẽ được xem 1 đoạn cutscene sau đó sẽ được chuyển đến map city, nơi người chơi tương tác với NPC Katherin và chuyển đến map tiếp theo.

Màn 1:

Tóm tắt: tầng đầu tiên của Dungeon, một hầm ngục cơ bản với cạm bẫy và quái vật.

Địa điểm: Tầng 1.

Mục tiêu: tiêu diệt hết các con quái có trong map, tìm cửa xuống tầng 2.

Màn 2:

Tóm tắt: Khu vực phía trên rừng cổ có những cây đại thụ chọc trời, hồ nước độc, được kết nối với nhau bằng những cây cầu treo.

Địa điểm: Tầng 2.

Mục tiêu: tiêu diệt hết các con quái có trong map, tìm cửa xuống tầng 3.

Màn 3:

Tóm tắt: Màn cuối cùng của game,, nơi trú ngụ của Boss cuối game Satan, xung quanh là cây cối rừng rậm bao vây không lối thoát.

Địa điểm: Tầng 3.

Mục tiêu: tiêu diệt Boss cuối giành chiến thắng.

Chương 4: Xây dựng Game và Demo

I. Xử lý di chuyển, va chạm, quái, kết thúc màn chơi:

- Di chuyển: Người chơi sử dụng bàn phím để di chuyển với tổ hợp phím WASD (tương ứng với tiến lên, sang trái, lùi xuống và sang phải), ngoài ra

```
private void FixedUpdate()
{
    if (canMove)
    {
        if (!isDashing)
        {
            // If movement input is not 0, try to move
            Vector2 desiredPosition = rb.position + movementInput * moveSpeed * Time.fixedDeltaTime;
            Vector2 smoothMovement = Vector2.Lerp(rb.position, desiredPosition, 0.8f); // Add smoothing
            bool success = TryMove(smoothMovement);
            animator.SetBool("isMoving", success);
        }
        else
        {
            animator.SetBool("isMoving", false);
        }
    }

    // Set direction of sprite to movement direction
    if (movementInput.x < 0)
    {
        spriteRenderer.flipX = true;
    }
    else if (movementInput.x > 0)
    {
        spriteRenderer.flipX = false;
    }
}
```

```
IEnumerator Dash()
{
    if (audioManager != null)
    {
        AudioManager.PlaySFX(audioManager.Player_Dash);
    }
    isDashing = true;
    animator.SetBool("isDashing", true);
    canMove = false;
    float startTime = Time.time;
    Physics2D.IgnoreLayerCollision(LayerMask.NameToLayer("PlayerLayer"), LayerMask.NameToLayer("EnemyLayer"), true);

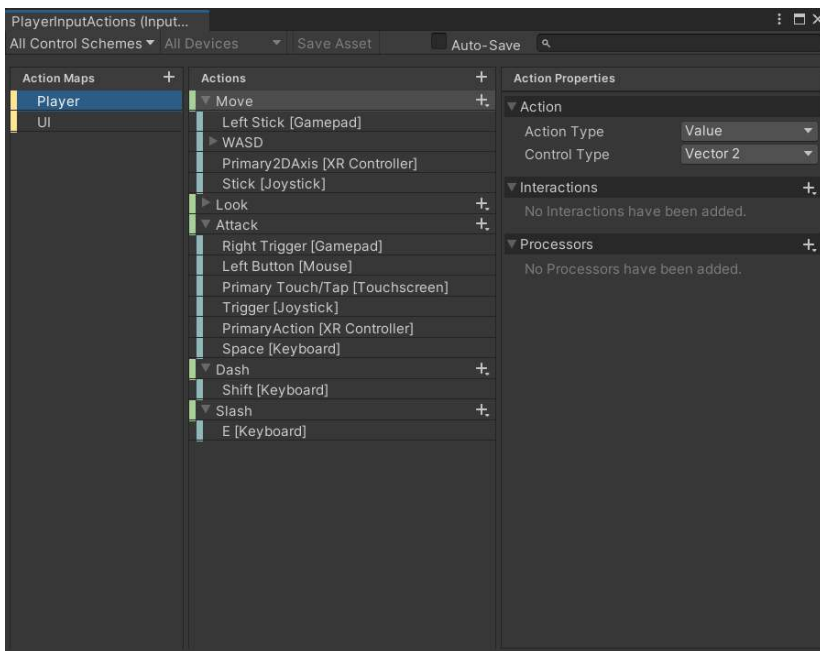
    while (Time.time < startTime + dashDuration)
    {
        rb.velocity = movementInput.normalized * dashSpeed;
        yield return null;
    }

    rb.velocity = Vector2.zero;
    Physics2D.IgnoreLayerCollision(LayerMask.NameToLayer("PlayerLayer"), LayerMask.NameToLayer("EnemyLayer"), false);
    isDashing = false;
    canMove = true;
    animator.SetBool("isDashing", false);

    // Apply cooldown
    isDashCooldown = true;
    yield return new WaitForSeconds(dashCooldown);
    isDashCooldown = false;
}
```

31

ấn nút Shift để thực hiện lướt. Nút F để tương tác với NPCs.



- Tấn công : Mouse 1 (chuột trái) để chém kiếm. Người ra người chơi có nút phụ là nút E để thực hiện tung đòn kỹ năng chém gió.

```

public void TriggerSlashEffect(bool isFacingLeft)
{
    if (!isOnCooldown)
    {
        isOnCooldown = true;
        gameObject.SetActive(true);

        gameObject.SetActive(true);

        float offsetX = isFacingLeft ? -9.92f : 9.92f; // Custom offset when facing right
        Vector3 localOffset = new Vector3(offsetX, 0.8f, 0f);
        transform.localScale = new Vector3(isFacingLeft ? -1.5f : 1.5f, 1.5f, 1.5f);

        // Calculate the world position based on the parent's position and local offset
        if (transform.parent != null)
        {
            transform.position = transform.parent.position + (transform.parent.localScale.x * localOffset);
        }
        else
        {
            transform.localPosition = localOffset; // Fallback if there is no parent
        }
        if (slashCollider != null)
        {
            slashCollider.enabled = true; // Enable the collider when the slash is active
        }

        animator.Play("Skill1", 0, 0f);
        Invoke("ResetCooldown", cooldownDuration);
        cooldownUI.StartCooldown(cooldownDuration);
    }
}

0 references
private void ResetCooldown()
{
    isOnCooldown = false;
    gameObject.SetActive(false);

    if (slashCollider != null)
    {
        slashCollider.enabled = false; // Disable the collider when the slash is done
    }
}

0 references
private void OnTriggerEnter2D(Collider2D other)
{
    IDamageable damageable = other.GetComponent<IDamageable>();

    if (damageable != null)
    {
        // Calculate knockback direction based on the slash effect's parent (likely the player)
        Vector3 parentPosition = transform.parent.position;
        Vector2 direction = (other.transform.position - parentPosition).normalized;

        damageable.OnHit(damage, direction * knockbackForce);
    }
}

```

- Quái được gán AEnemy để chúng tự động đi săn lùng mục tiêu, khi thấy người chơi nó sẽ tiến lại gần và tấn công, khi gặp các chướng ngại vật chúng sẽ tìm hướng đi khác. Slime có 5 đơn vị máu, Pig có 3 đơn vị máu, Boss có 20 đơn vị máu, với mỗi phát chém từ kiếm của người chơi chúng giảm 1 máu, dùng kĩ năng sẽ giảm 2 máu. Quái chết khi máu < 0 và gọi tới animator nằm xuống, sau một khoảng thời gian xác sẽ tự biến mất. Quái sẽ có sát thương là 1.

```

// reference
void MoveTowardsPlayer()
{
    if (player != null)
    {
        patrolBehavior.enabled = false;
        Vector2 direction = (player.position - transform.position).normalized;
        rb.MovePosition((Vector2)transform.position + (direction * moveSpeed * Time.deltaTime));

        // Flip sprite based on movement direction
        if (direction.x < 0)
        {
            spriteRenderer.flipX = true;
        }
        else if (direction.x > 0)
        {
            spriteRenderer.flipX = false;
        }
    }
}

```

```

// reference
private void Attack()
{
    lastAttackTime = Time.time;
    // Trigger attack through the Enemy_Attack script
    if (spriteRenderer.flipX == true)
    {
        attackScript.AttackLeft();
        animator.SetTrigger("Attack");
    }
    else
    {
        attackScript.AttackRight();
        animator.SetTrigger("Attack");
    }

    StartCoroutine(DisableAttackCollider());
}

```

- Thanh máu: Máu là một giá trị quan trọng với người chơi, luôn luôn phải giữ cho máu đầy nếu không thì trò chơi sẽ kết thúc (Game Over). Nó sẽ được hiển thị ở góc trái phía bên trên của màn hình.

```
public float Health
{
    set
    {
        currentHealth = Mathf.Clamp(value, 0, maxHealth);
        if (currentHealth > 0)
        {
            animator.SetTrigger("IsAttacked");
            if (audioManager != null)
            {
                audioManager.PlaySFX(audioManager.Player_GetHit);
            }
            IsAttacked = true;
            playerController.DisableInput();
        }
        if (currentHealth <= 0)
        {
            Defeated();
        }
    }
    get
    {
        return currentHealth;
    }
}
```

- Chuyển scene:

```
private void Start()
{
    // Ensure SceneManagement and PlayerController exist and are initialized
    if (SceneManagement.Instance != null && PlayerController.Instance != null && transitionName == SceneManagement.Instance.SceneTransitionName)
    {
        PlayerController.Instance.transform.position = transform.position;
        class CameraController ntroller (use null-conditional operator)
        CameraController.Instance?.SetPlayerCameraFollow();
    }
    else
    {
        // Add logging to help diagnose potential issues
        Debug.LogWarning("AreaEntrance: SceneManagement, PlayerController, or CameraController not found or conditions not met.");
    }
}
```

```
1 reference
[SerializeField] private string sceneToLoad;
1 reference
[SerializeField] private string sceneTransitionName;

0 references
private void OnCollisionEnter2D(Collision2D collision)
{
    if (collision.gameObject.GetComponent<PlayerController>())
    {
        LeanTween.cancelAll();
        SceneManager.LoadScene(sceneToLoad);
        SceneManagement.Instance.SetTransitionName(sceneTransitionName);
    }
}
```

- Khi người chơi hết máu thì trò chơi sẽ kết thúc (game Over).
- Âm thanh: bao gồm các sound cơ bản của trò chơi.

```

0 references
void Start()
{
    musicSource = GetComponent();
    PlayNormalBackgroundMusic();
}

2 references
public void PlayNormalBackgroundMusic()
{
    musicSource.clip = background;
    musicSource.loop = true;
    musicSource.Play();
}

1 reference
public void PlayEnemyBackgroundMusic()
{
    musicSource.clip = enemyBackgroundMusic;
    musicSource.loop = true;
    musicSource.Play();
}

13 references
internal void PlaySFX(AudioClip clip)
{
    if (SFXSource && clip) // Check if SFXSource and clip are not null
    {
        SFXSource.PlayOneShot(clip);
    }
}

```

II. Demo:

2.1. Gamplay:

- Bắt đầu người chơi đánh Boss Rồng và thất bại, sau đó được hồi sinh tại cổng hồi sinh ở thành phố.
- Mục tiêu chính: Sống sót và "Phá đảo" khi đánh bại "Boss cuối".
- Người chơi dùng chuột để định hướng di chuyển. Dùng phím W, A, S, D để đi chuyển, phím Shift để chạy nhanh.
- Khi kiểm chạm vào cơ thể quái vật sẽ mất 1 máu. Sử dụng kỹ năng sẽ mất 3 máu, tuy nhiên chúng vẫn tấn công người chơi dù bị tấn công.
- Số lượng quái vật nằm rải rác khắp bản đồ, người chơi có thể "lạc" vào khu vực trùm (boss).
- Khi người chơi đi xuống tầng sâu nhất sẽ đến khu vực gặp Boss cuối, người chơi mất máu nhiều hơn thì cơ hội sống của người chơi càng thấp. Tiêu diệt Boss người chơi sẽ chiến thắng. Nếu bị tiêu diệt người chơi sẽ thua và quay trở lại màn hình chính.

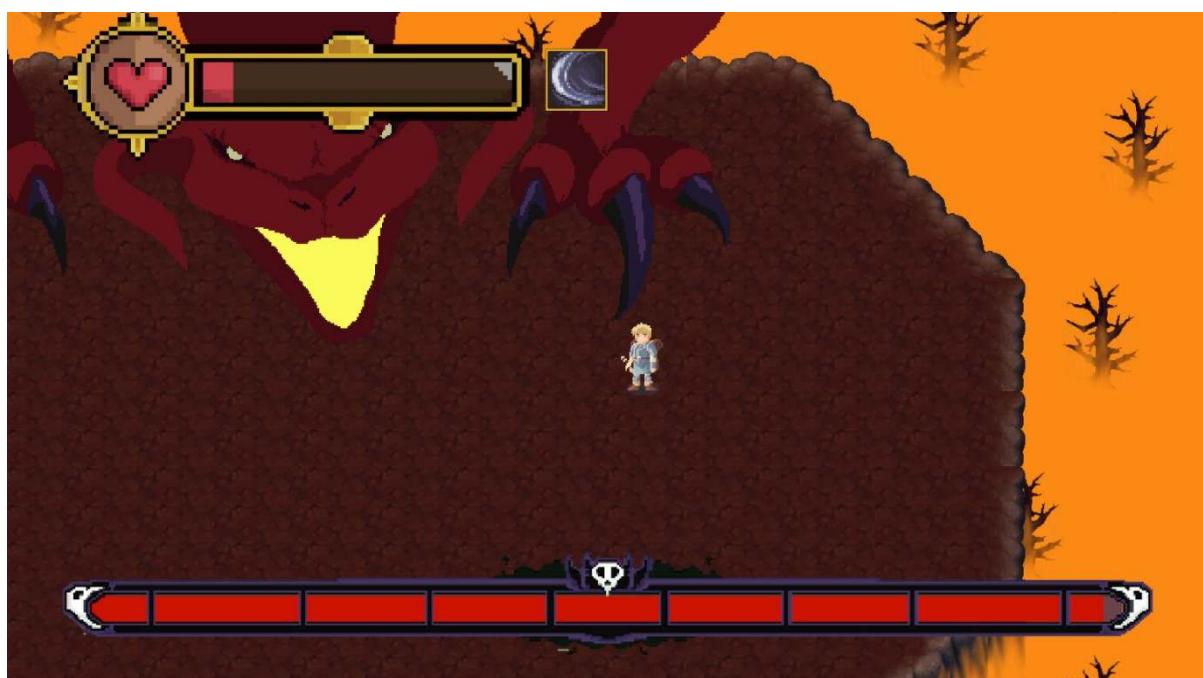
2.2 Demo trò chơi:



Hình 4.4: Main Menu



Hình 4.5: Intro



Hình 4.6: Màn chiến đấu với Boss Rồng



Hình 4.7: Người chơi tương tác với NPC



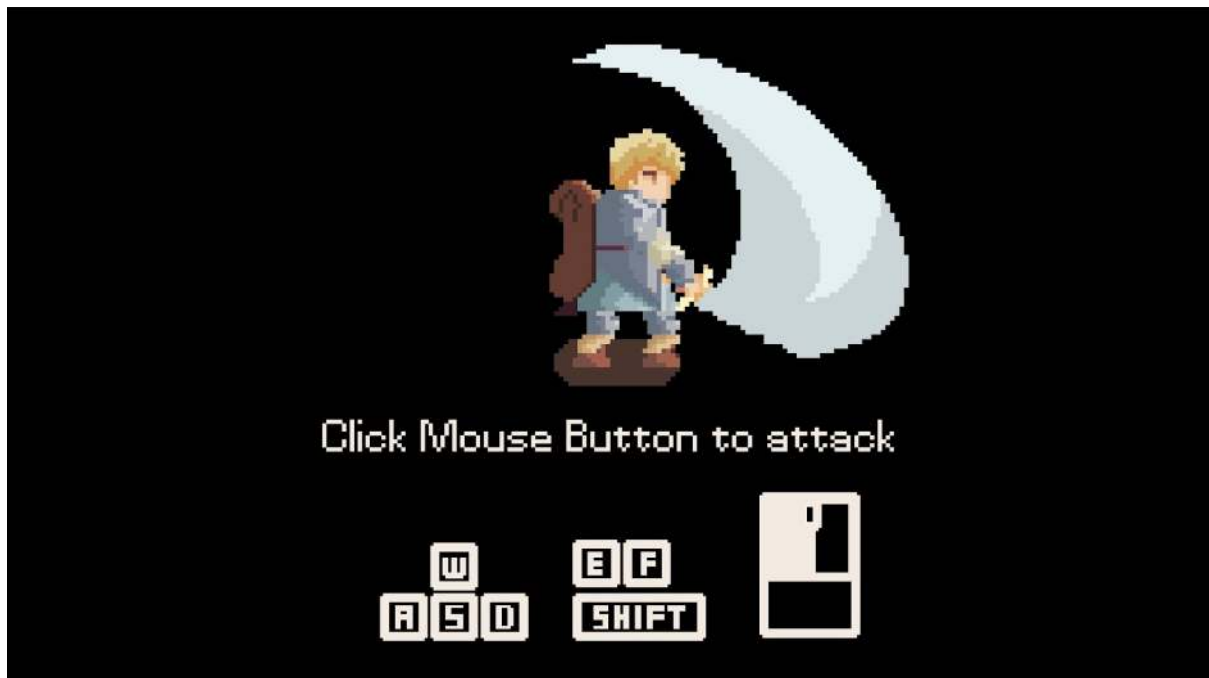
Hình 4.8: Màn 1



Hình 4.9: Màn 2



Hình 5: Màn 3



Hình 5.1: Hướng dẫn chơi



Hình 5.3: Màn hình GameOver

III. Các chức năng cơ bản:

Các tính năng chính được đưa ra và test trong demo bao gồm:

- Chức năng cơ bản của player (chém, sử dụng kỹ năng, lướt, di chuyển...).
- Chức năng kẻ địch (di chuyển, tìm đường đến player, tấn công player khi vào tầm với, bị hạ gục...).
- Chức năng hiển thị của map (ánh sáng, vật dụng và quang cảnh).

- Chức năng nhạc nền, và GameOver.
- Hiện tại trong trường trình demo, vũ khí đã được trang bị sẵn và không cần phải mua. có những setting riêng biệt, âm thanh, vv..

Chương 5: Kết luận và định hướng phát triển

I. Kết quả đạt được:

- Game có giao diện thân thiện, đẹp mắt, dễ sử dụng.
- Game đã đạt được yêu cầu cơ bản của trò chơi.
- Ưu điểm:
 - Hiểu được cấu trúc, cách thức hoạt động, cách thức phát triển ứng dụng trên
 - Nâng cao kiến thức lập trình và tiếng anh chuyên ngành thông qua những tài liệu về Unity bằng tiếng anh.
 - Tổ chức load các màn tương ứng thông qua các button.
 - Xây dựng các chức năng như điểm, màn chơi thông qua ngôn ngữ lập trình C#.
 - Tạo âm thanh cho trò chơi và 1 số chức năng khác.
- Nhược điểm:
 - Do kiến thức của em về Unity còn hạn hẹp nên các chức năng trong game còn nghèo nàn.
 - Chưa xây dựng được hoàn chỉnh 1 game như mong muốn.

II Kết luận:

Qua trò chơi demo chúng em đã từng bước tiếp cận được với công nghệ làm game, cách thức cũng như phương pháp, nền tảng tối ưu, biết cách kết hợp các phần mềm để tạo ra một mô hình vật thể, sản phẩm. Trong quá trình xây dựng game chúng em nhận ra còn nhiều vấn đề phát sinh, nhiều khó khăn hơn mình suy nghĩ, rất nhiều bug trong code C# cần fix, đôi khi 1 lỗi nhỏ làm hỏng cả hệ thống, vì vậy khi thiết kế game chúng ta nên chia nhỏ từng công việc và module riêng ra để dễ xây dựng và gắn kết khi các thành phần đã chạy ổn định. Viết game rất tốn thời gian cho các công việc như design các nhân vật, bản đồ, đội ngũ xây dựng phải đông đảo hơn, cần sự kiên trì và sáng tạo hơn.

III. Định hướng phát triển:

- Thiết kế thêm và hoàn thiện về nhân vật (vũ khí và các nâng cấp).
- Thiết kế thêm và thêm chức năng cho Enemy (phù hợp với bản đồ).
- Thiết kế thêm các màn chơi.
- Chức năng unlock vũ khí, điều chỉnh độ khó, lưu và tải dữ liệu của màn chơi.
- Các chức năng điều chỉnh options (âm thanh, ánh sáng).
- Thêm các chức năng rơi vật phẩm từ quái, nhặt vật phẩm, kho đồ, hệ thống kĩ năng nhân vật, vv...
- Tối ưu hóa các xử lý giúp game chạy nhanh hơn.

Bảng phân chia công việc

Họ và tên	Mã sinh viên	Công việc	Tiến độ hoàn thành
Nguyễn Tài Thu	2155020092	Code chính, xây dựng toàn bộ game	100%
Trần Việt Hoàng	2155020034	Animator nhân vật, NPCs, thiết kế UI cho game, làm Tutorial.	100%
Mai Ngọc Quyến	2155020080	Animator cho boss, vẽ map và cắt cảnh, làm hội thoại trong game, thuyết trình.	100%
Phạm Đức Trung	2155020104	Code phụ, animator cho quái, viết báo cáo.	100%
Phạm Thế Anh	2155020008	Làm Audio, animator cho quái, làm powerpoint.	100%